TASK 1

Capabilities the system have :-

- **manageProfile** -- [Manage profile and addresses]
- **browseCatalogue** -- [List restaurants near the user and show a restaurant's menu with prices]
- **manageCart** -- [Add/remove items from user's cart before checkout]
- **placeOrder** -- [Convert a cart into a confirmed order: Validate items, take payment, assignment delivery]
- **trackOrder** -- [Let a user check current order status, or cancel before it ships]
- **processPayment** -- [Charge the user for an order and issue refunds if needed]
- **assignAndTrackDelivery** -- [Pick a rider for an order and update delivery status until dropped off]
- **sendNotification** -- [Send a status message to the user at each milestone and]

TASK 3

Contract :- Accounts

Operation	Input	Output	Errors
register	name, email, password	studentId, status	Email already registered, Invalid email/password
login	email, password	studentId, sessionToken	Invalid credentials
addAddress	studentId, label, addressText	addressId	Invalid address
getAddress	addressId, studentId	addressText, label	Not found, address does not belong to student

Contract :- Catalogue

Operation	Input	Output	Errors
listRestaurants	area / filter	restaurants[]	Restaurant unavailable
getMenu	restaurantId	items[] (itemId, name, price)	Restaurant not found
checkItem	itemId	available (bool), price	Item not found

Contract :- Orders

Operation	Input	Output	Errors
addToCart	studentId, itemId, qty	cartSummary	Item unavailable
placeOrder	studentId, items[{itemId, qty}], deliveryAddressId, paymentMethodId	orderId, status, total, estimatedMinutes	Empty cart, item unavailable, invalid address, payment declined, no rider available
getOrder	orderId	orderDetails, orderStatus	Order not found
cancelOrder	orderId	orderStatus	Too late to cancel

Contract :- Payments

Operation	Input	Output	Errors
charge	studentId, orderId, amount, paymentMethodId	transactionId, receipt	Payment declined, invalid payment method
refund	transactionId	refundId, status	Transaction not found, already refunded

Contract :- Delivery

Operation	Input	Output	Errors
assignRider	orderId, deliveryAddressId	assignmentId, riderName, eta	No rider available
getDeliveryStatus	orderId	status, lastKnownLocation	Assignment not found

Contract :- Notifications

Operation	Input	Output	Errors
send	studentId, orderId, messageType	notificationId, status	Invalid recipient
getMessageStatus	studentId, orderId	status	Message not found

TASK 4

placeOrder Operation Specification :-

Orders services -

Inputs

Field	Type	Required	Description
userReference	Reference (ID)	Yes	User placing the order
restaurantReference	Reference (ID)	Yes	Restaurant the order is placed with
items	List of (menuItemReference, quantity)	Yes, min 1	Menu items and quantities ordered
deliveryAddressReference	Reference (ID)	Yes	User's saved address to deliver to

Outputs

Field	Type	Description
orderReference	Reference (ID)	Unique ID for the new order
orderSummary.totalAmount	Reference (ID)	Restaurant the order is placed with
orderSummary.status	Status label	Initial order status
orderSummary.createdTime	Timestamp	Time the order was created

Error Cases

Error Code	When It Occurs
EMPTY_CART	Items list is empty
ITEM_UNAVAILABLE	One or more items unavailable / out of stock / not on menu
RESTAURANT_CLOSED	Address reference invalid or not owned by user
INVALID_ADDRESS	Timestamp
PRICE_MISMATCH	Expected price does not match current menu price

Internal Details Hidden From Callers

Detail	What is Hidden
Data storage & schema	Orders/carts table structure, keys, indexes, how orderReference is generated, audit/history logs
Price calculation	How Catalogue prices are looked up and summed; caching of menu/price data
Cross-service calls	Sequence/number of calls to Catalogue (checkItem) and Accounts (getAddress); retry/timeout/fallback logic
Stock locking	Internal reservation of menu-item stock to prevent over-selling on concurrent orders
Order state machine	Full set of internal statuses and allowed transitions between them
Idempotency & transactions	Deduplication of retried requests; transaction boundaries for writing the order
Logging & observability	Internal logs, metrics, and traces used for monitoring/debugging

TASK 6

1. User Service

Property	Result	Fix / Action	Could Fail If
Reachable	Pass	Expose account operations through its service interface so other services can reach user and address information.	The service endpoint is unavailable, overloaded, or authentication operations cannot be reached.
Self-contained	Pass	No change needed — it owns users and delivery addresses within its own service boundary.	Another service requires direct access to the User Service database to perform account or address operations.
Has a contract	Pass	Formalize authenticateUser, getUser, and getAddress request, response, and error formats.	The API behavior or error responses are undocumented or change without versioning.
Independent	Pass	Keep user and address storage under the User Service and avoid depending on another service's database.	Account operations cannot function because they require direct database access to another service.
Loosely coupled	Pass	Use APIs or events for communication; never expose the User Service database to other services.	Orders or another service reads the users/address tables directly.

2. Catalogue Service

Property	Result	Fix / Action	Could Fail If
Reachable	Pass	Expose listRestaurants, getMenu, and checkItem through a reachable service API.	High read load during peak ordering hours overwhelms the service if it is not scaled or cached properly.
Self-contained	Pass	No change required — it owns merchants and menu items.	Another service must directly query catalogue tables to obtain restaurant or menu information.
Has a contract	Pass	Formalize request, response, and error formats for catalogue operations.	Consumers depend on undocumented response structures or inconsistent error codes.
Independent	Pass	Keep merchant and menu-item storage inside the Catalogue Service.	Catalogue operations depend on another service's database instead of its own data.
Loosely coupled	Pass	Do not allow other services to directly access the Catalogue database; use APIs or events.	Orders or another service directly reads or writes catalogue tables.

3. Order Service

Property	Result	Fix / Action	Could Fail If
Reachable	Pass	Expose placeOrder, getOrder, cancelOrder, and updateStatus through the service contract.	The Order Service is unavailable or its dependencies make the operation effectively unreachable.
Self-contained	Pass	Keep orders, order items, and order state inside the Order Service; use references for external entities.	The Order Service stores or manipulates users, merchants, or catalogue records directly.

Property	Result	Fix / Action	Could Fail If
Has a contract	Pass	Use the defined placeOrder/getOrder/cancelOrder/updateStatus operations with documented inputs, outputs, and errors.	Callers need to know internal order tables, state-machine details, or storage mechanics.
Independent	Pass	Own order data and use service calls to Catalogue and User services rather than direct database access.	The Order Service cannot operate without another service's database being available or directly accessible.
Loosely coupled	Pass	Communicate with User and Catalogue services through their contracts; keep cross-service identifiers as references.	The Order Service directly joins or modifies User/Catalogue database tables.

4. Payment Service

Property	Result	Fix / Action	Could Fail If
Reachable	Pass	Expose authorizePayment, capturePayment, and refundPayment through the payment contract.	The upstream third-party payment gateway is down or rate-limiting, making payment operations effectively unreachable.
Self-contained	Pass	No change required — it owns payment transactions and refunds.	Payment records or refund logic are moved into another service's database.
Has a contract	Pass	Formalize request, response, and error formats for payment operations.	Payment operations change without a stable contract or documented error behavior.
Independent	Pass	Manage payment and refund data within the Payment Service without directly depending on another service's database.	It requires direct access to Orders or Users database tables.
Loosely coupled	Pass	Do not allow direct access to Orders or Users databases; communicate through APIs or events.	Payment processing is implemented through direct database reads/writes across service boundaries.

5. Delivery Service

Property	Result	Fix / Action	Could Fail If
Reachable	Pass	Expose delivery assignment and status operations through the Delivery Service contract.	Rider/GPS integration is unavailable or the service cannot handle a spike in delivery requests.
Self-contained	Pass	No change required — it owns delivery assignments and delivery status.	Delivery operations require direct access to Order Service tables.
Has a contract	Pass	Use assignRider, getAssignment, and updateDeliveryStatus with defined inputs, outputs, and errors.	The delivery API is undocumented or callers depend on internal delivery storage.
Independent	Pass	Keep delivery data inside the Delivery Service and use references to orders, merchants, addresses, and couriers.	It directly reads or writes another service's database to manage deliveries.
Loosely coupled	Pass	Communication with Order Service should happen through APIs or events, not direct database access.	Delivery and Order services share database tables or directly query each other's schemas.

6. Notification Service

Property	Result	Fix / Action	Could Fail If
Reachable	Pass	Expose sendNotification, getMessageStatus, and logMessage through the Notifications Service interface.	The Notifications Service endpoint is unavailable, overloaded, or notification requests cannot be reached.
Self-contained	Pass	Own the message_log data and notification processing inside the Notifications Service.	Another service requires direct access to the message_log database to create or update notifications.
Has a contract	Pass	Formalize sendNotification, getMessageStatus, and logMessage request, response, and error formats.	Consumers depend on undocumented notification responses or notification behavior changes without versioning.
Independent	Pass	Keep message_log storage and notification logic inside the Notifications Service; use service contracts for incoming requests.	Notification processing requires direct access to Order or Delivery database tables.
Loosely coupled	Pass	Receive notification requests through service contracts or events; do not expose message_log to other services.	Order or Delivery services directly read or write message_log tables.

----X-X-X-X----